DCU模型适配通用流程

📌 量化模型情况较为复杂，该文档仅针对普通模型适配。

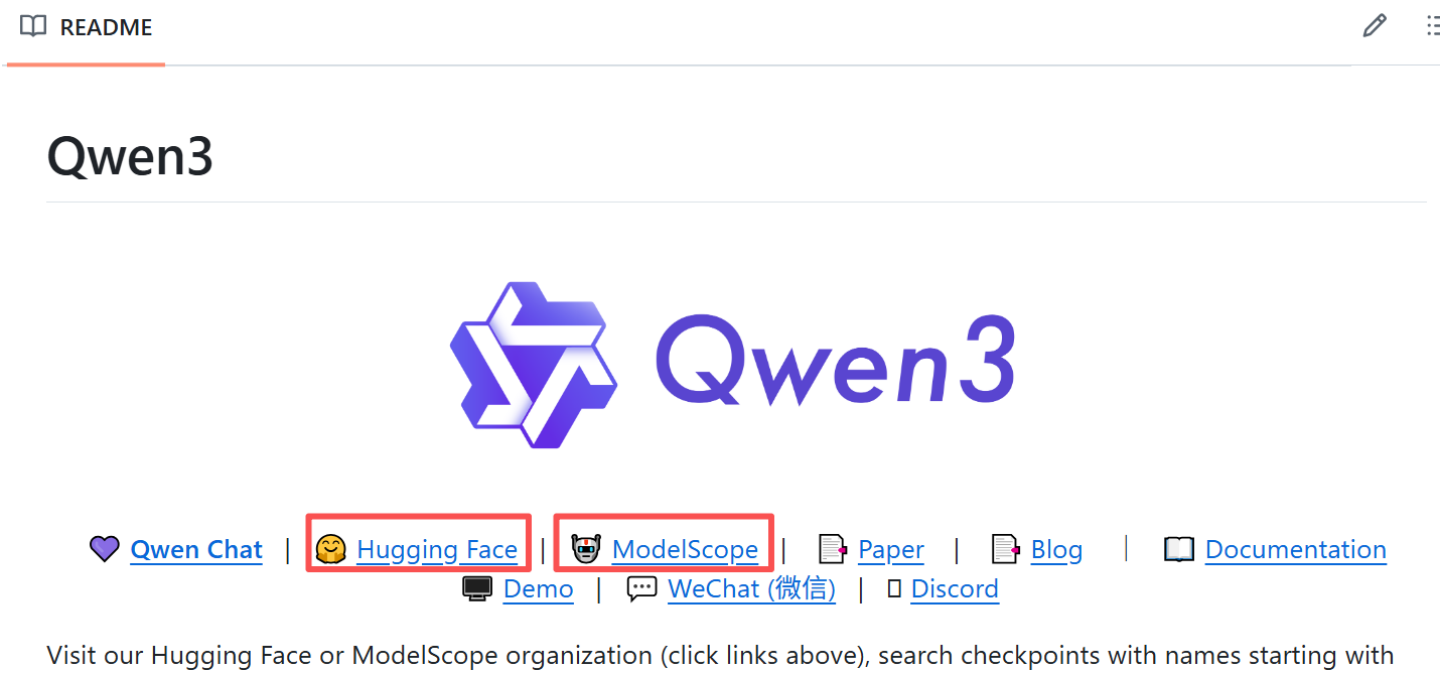
1 模型准备

模型下载可参考官方发布的 Blog、GitHub 仓库以及 Documentation 等技术文档，获取官方推荐的下载方式（支持在线或离线下载）及推荐的稳定版本。同时，也可根据实际业务需求选择合适的模型版本进行下载。

下载前，建议根据目标运行环境，先初步确认所需深度学习框架（如 PyTorch、vLLM、PaddlePaddle 等）及其对应版本在 DCU上的支持情况与兼容性。

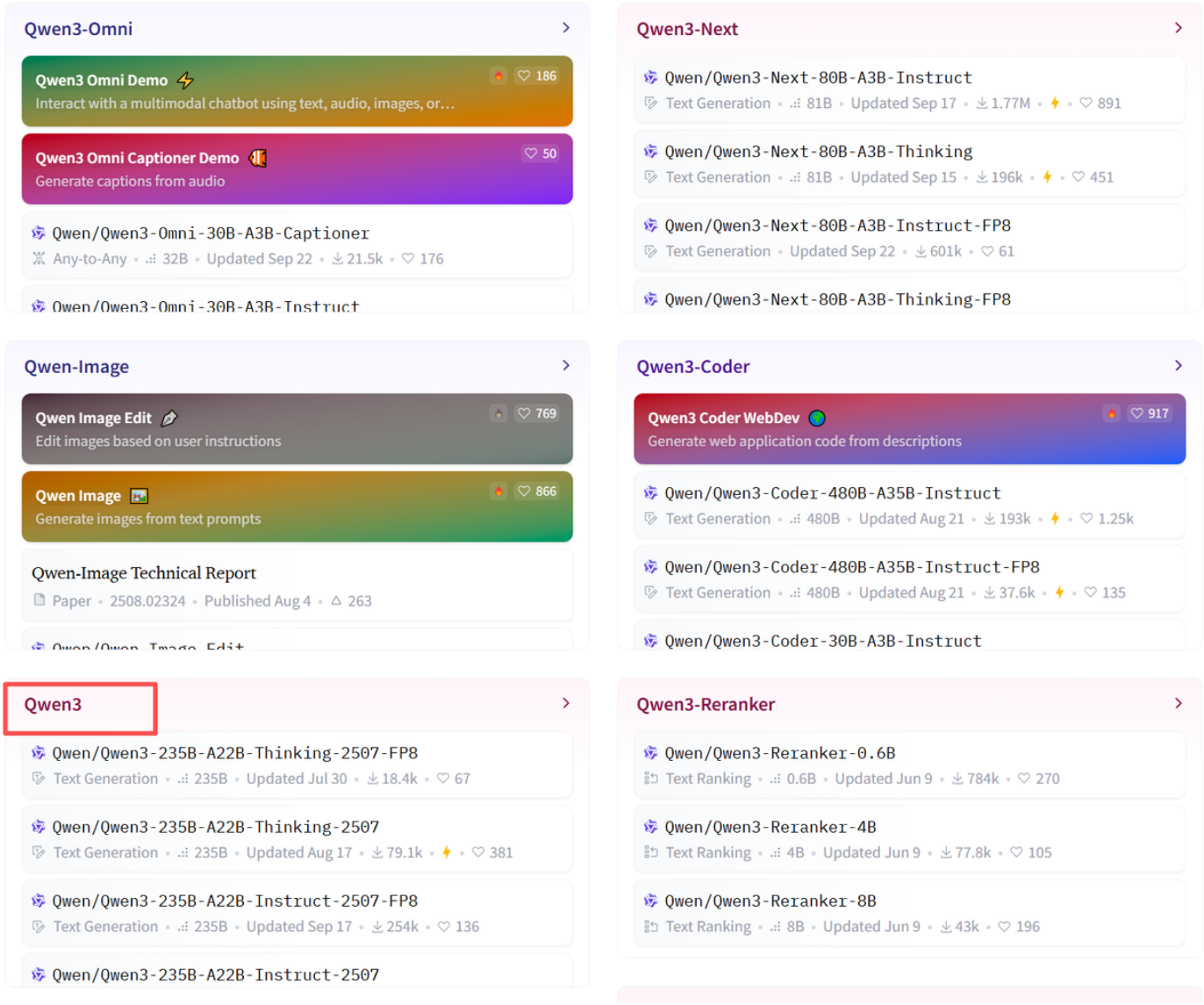
- **以Qwen3为例：**

（1）进入Qwen3的官方仓库：https://github.com/QwenLM/Qwen3

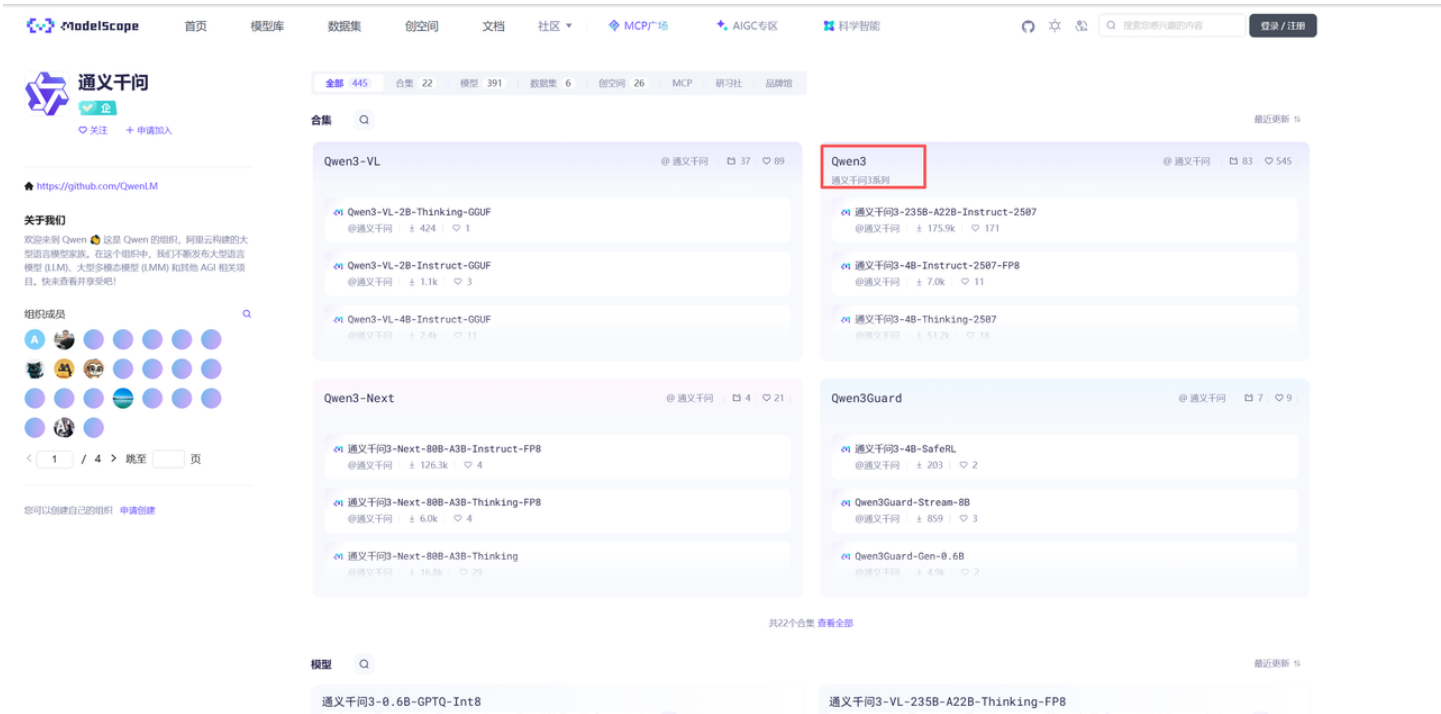


（2）查看模型下载地址，可通过Hugging Face或者ModelScope下载：

- Hugging Face官方源：https://huggingface.co/Qwen
- 由于Hugging Face可能需要外网才能获取，推荐使用国内镜像：https://hf-mirror.com/Qwen



- ModelScope平台：https://modelscope.cn/organization/qwen



2 模型训练和推理

2.1 训练

对于大模型训练，DCU 平台训练框架支持现状及评估方法如下：

2.1.1 已验证良好支持的框架

- LLaMA-Factory（全功能支持，包括 LoRA、QLoRA、DPO 等）
- Megatron-LM + DeepSpeed（支持多节点分布式训练）
- MS SWIFT

2.1.2 其他框架支持性快速判断方法

首先检查项目依赖文件（requirements.txt、setup.py、pyproject.toml、environment.yml 等）：

1. 若依赖主要为以下通用组件 `transformers 、accelerate、peft、deepspeed、bitsandbytes、datasets` 等，可在 DCU 上快速移植。
2. 若出现以下情况，需提前与平台技术支持确认：
   - 依赖特定版本 Triton 或自定义 CUDA 扩展
   - 使用 vLLM、FlashAttention-2 等对底层算子高度优化的推理框架
   - 包含非 PyTorch 原生实现的重算子（如 xformers、Apex 等）

以下是llama-factory的依赖样例，无特殊依赖库，可直接安装使用：

LLaMA-Factory / requirements.txt

hiyouga [deps] fix yanked packages (#9333) ✓

Code Blame 38 lines (38 loc) · 590 Bytes

```
# core deps
transformers>=4.49.0,<=4.56.2,!=4.52.0; python_version < '3.10'
transformers>=4.49.0,<=4.57.1,!=4.52.0,!=4.57.0; python_version >= '3.10'
datasets>=2.16.0,<=4.0.0
accelerate>=1.3.0,<=1.11.0
peft>=0.14.0,<=0.17.1
trl>=0.8.6,<=0.9.6
# gui
gradio>=4.38.0,<=5.45.0
matplotlib>=3.7.0
tyro<0.9.0
# ops
einops
numpy<2.0.0
pandas>=2.0.0
scipy
# model and tokenizer
sentencepiece
tiktoken
modelscope>=1.14.0
hf-transfer
safetensors<=0.5.3
# python
fire
omegaconf
packaging
protobuf
pyyaml
pydantic<=2.10.6
# api
uvicorn
fastapi
sse-starlette
# media
av
librosa
# yanked
propcache!=0.4.0
```

2.2 推理

2.2.1 Transformers推理

GPGPU架构的DCU对于transformers可以做到完全兼容，一般情况下无需特殊适配。

如果模型官方支持transformers框架的推理，在DCU上安装模型推理所对应的transformers版本、torch（DCU 适配版）和其他必要依赖，最后将模型地址按照需求修改为自己的存放地址，则可直接进行推理。

此外，transformers未对显存进行优化，推理过程中注意合理安排显存资源。

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_name = "Qwen/Qwen3-235B-A22B-Thinking-2507"

# load the tokenizer and the model
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto"
)

# prepare the model input
prompt = "Give me a short introduction to large language model."
messages = [
    {"role": "user", "content": prompt}
]
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
)
model_inputs = tokenizer([text], return_tensors="pt").to(model.device)

# conduct text completion
generated_ids = model.generate(
    **model_inputs,
    max_new_tokens=32768
)
output_ids = generated_ids[0][len(model_inputs.input_ids[0]):].tolist()
```

```
# parsing thinking content
try:
    # rindex finding 151668 (</think>)
    index = len(output_ids) - output_ids[::-1].index(151668)
except ValueError:
    index = 0

thinking_content = tokenizer.decode(output_ids[:index], skip_special_tokens=True)
content = tokenizer.decode(output_ids[index:], skip_special_tokens=True).strip("\

print("thinking content:", thinking_content) # no opening <think> tag
print("content:", content)
```

2.2.2 vllm推理

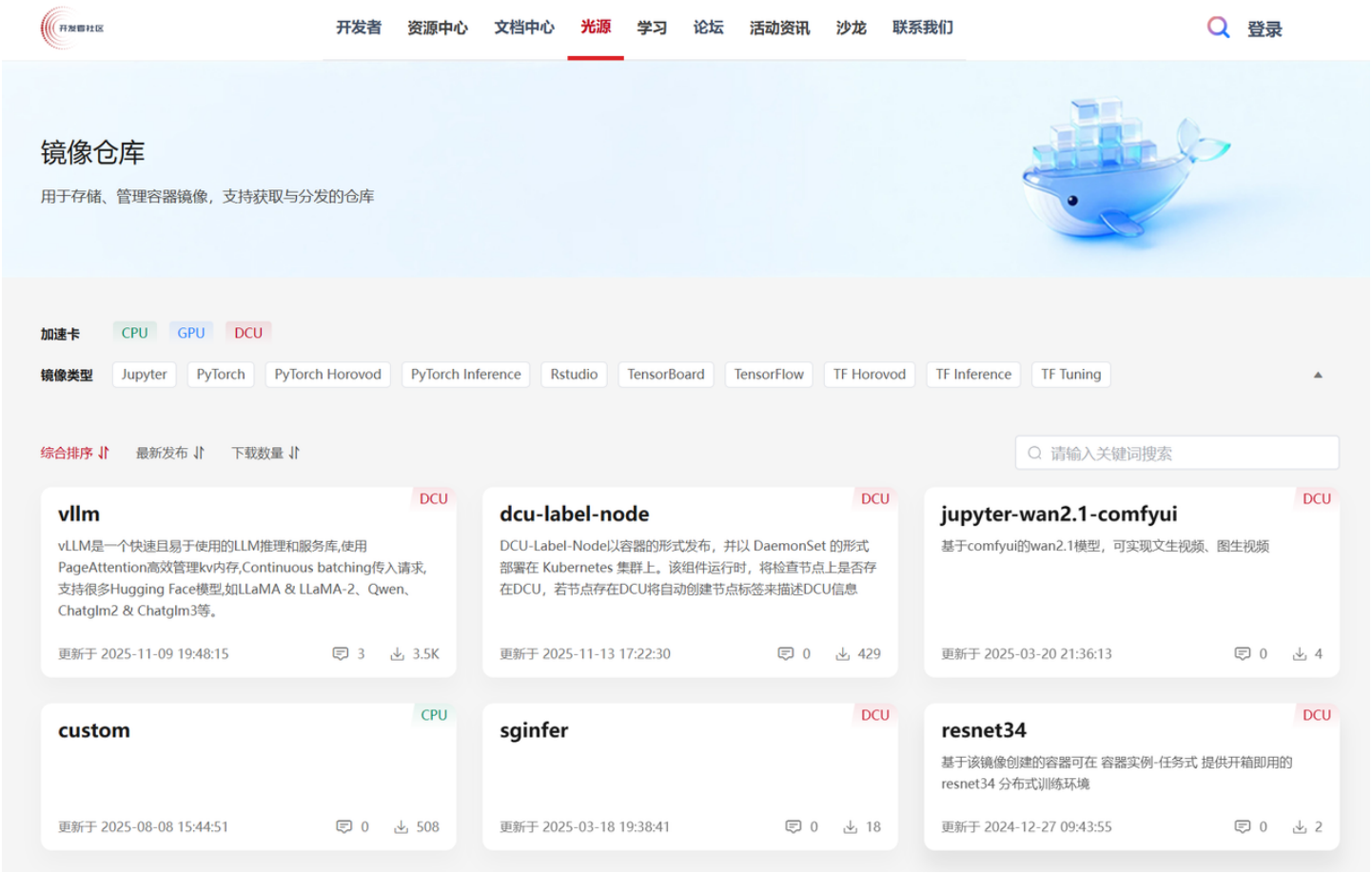
2.2.2.1 vllm 安装

vLLM 已在 DCU 平台完成深度兼容性适配，支持高吞吐、低延迟的连续批处理推理。使用前需安装 DCU 专用版本的 vLLM 包。

- **安装包下载：**访问光合开发者社区（https://developer.sourcefind.cn/tool/ ），搜索并下载与当前平台版本匹配的 vLLM wheel 包及其配套深度学习依赖包，确保版本完全对齐，避免兼容性问题。



- **镜像环境：**使用社区光源（https://developer.sourcefind.cn/servicelist）提供的镜像源，直接拉取已集成 DCU 适配版 vLLM 的完整环境（推荐用于快速验证和部署）。



2.2.2.2 vllm支持情况

1. vllm的环境安装好后，可以通过`pip show vllm`查看当前vllm包所在地址：

```
root@bw53:/workspace# pip show vllm
Name: vllm
Version: 0.11.0+das.opt1.alpha.dtk25042
Summary: A high-throughput and memory-efficient inference and serving engine for LLMs
Home-page: https://github.com/vllm-project/vllm
Author: vLLM Team
Author-email:
License-Expression: Apache-2.0
Location: /usr/local/lib/python3.10/dist-packages
Requires: aiohttp, blake3, boto3, botocore, cachetools, cbor2, cloudpickle, cmake, compressed-
mla, gguf, lark, lightop, llguidance, lm-format-enforcer, lmslim, mistral_common, msgspec, nin
ore, packaging, partial-json-parser, peft, pillow, prometheus-fastapi-instrumentator, promethe
n-json-logger, pytrie, pyyaml, pyzmq, ray, regex, requests, runai-model-streamer, runai-model-
rizer, tiktoken, timm, tokenizers, torch, tqdm, transformers, triton, typing_extensions, watch
```

2. 打开上述**vllm安装地址下的**`vllm/model_executor/models/registry.py`代码，搜索是否存在模型文件-config.json文件中的**"architectures"字段内容下的模型类别。**如果存在，则可以支持该模型的基础推理。如果不存在，则不支持，请联系相关DCU开发人员提需求。

```
"PhiMoEForCausalLM": ("phimoe", "PhiMoEForCausalLM"),
"Plamo2ForCausalLM": ("plamo2", "Plamo2ForCausalLM"),
"QWenLMHeadModel": ("qwen", "QWenLMHeadModel"),
"Qwen2ForCausalLM": ("qwen2", "Qwen2ForCausalLM"),
"Qwen2MoeForCausalLM": ("qwen2_moe", "Qwen2MoeForCausalLM"),
"Qwen3ForCausalLM": ("qwen3", "Qwen3ForCausalLM"),
"Qwen3MoeForCausalLM": ("qwen3_moe", "Qwen3MoeForCausalLM"),
"SeedOssForCausalLM": ("seed_oss", "SeedOssForCausalLM"),
"Step3TextForCausalLM": ("step3_text", "Step3TextForCausalLM"),
"StableLMEpochForCausalLM": ("stablelm", "StablelmForCausalLM"),
"StableLmForCausalLM": ("stablelm", "StablelmForCausalLM"),
"Starcoder2ForCausalLM": ("starcoder2", "Starcoder2ForCausalLM"),
"SolarForCausalLM": ("solar", "SolarForCausalLM"),
"TeleChat2ForCausalLM": ("telechat2", "TeleChat2ForCausalLM"),
"TeleFLMForCausalLM": ("teleflm", "TeleFLMForCausalLM"),
"XverseForCausalLM": ("llama", "LlamaForCausalLM"),
"Zamba2ForCausalLM": ("zamba2", "Zamba2ForCausalLM"),
```

main Qwen3-235B-A22B-Thinking-2507 / config.json

littlebird13 Update config.json 37fe633 VERIFIED

raw Copy download link history blame contribute delete Safe

```
{
  "architectures": [
    "Qwen3MoeForCausalLM"
  ],
  "attention_bias": false,
  "attention_dropout": 0.0,
  "bos_token_id": 151643,
  "decoder_sparse_step": 1,
  "eos_token_id": 151645,
  "head_dim": 128,
  "hidden_act": "silu",
  "hidden_size": 4096,
  "initializer_range": 0.02,
  "intermediate_size": 12288,
  "max_position_embeddings": 262144,
  "max_window_layers": 94,
```

3. 查看是否支持工具调用

工具调用的支持情况可以查看 `vllm/entrypoints/openai/tool_parsers/__init__.py` 中是否有查询模型的类别。如果不存在，则不支持，请联系相关DCU开发人员提需求。

```
from .granite_tool_parser import GraniteToolParser
from .hermes_tool_parser import Hermes2ProToolParser
from .hunyuan_a13b_tool_parser import HunyuanA13BToolParser
```

```python
from .hunyuan_a13b_tool_parser import HunyuanA13BToolParser
from .internlm2_tool_parser import Internlm2ToolParser
from .jamba_tool_parser import JambaToolParser
from .kimi_k2_tool_parser import KimiK2ToolParser
from .llama4_pythonic_tool_parser import Llama4PythonicToolParser
from .llama_tool_parser import Llama3JsonToolParser
from .longcat_tool_parser import LongcatFlashToolParser
from .minimax_tool_parser import MinimaxToolParser
from .mistral_tool_parser import MistralToolParser
from .openai_tool_parser import OpenAIToolParser
from .phi4mini_tool_parser import Phi4MiniJsonToolParser
from .pythonic_tool_parser import PythonicToolParser
from .qwen3coder_tool_parser import Qwen3CoderToolParser
from .qwen3xml_tool_parser import Qwen3XMLToolParser
from .seed_oss_tool_parser import SeedOssToolParser
from .step3_tool_parser import Step3ToolParser
from .xlam_tool_parser import xLAMToolParser

__all__ = [
    "ToolParser",
    "ToolParserManager",
    "Granite20bFCToolParser",
    "GraniteToolParser",
    "Hermes2ProToolParser",
    "MistralToolParser",
    "Internlm2ToolParser",
    "Llama3JsonToolParser",
    "JambaToolParser",
    "Llama4PythonicToolParser",
    "LongcatFlashToolParser",
    "PythonicToolParser",
    "Phi4MiniJsonToolParser",
    "DeepSeekV3ToolParser",
    "DeepSeekV31ToolParser",
    "xLAMToolParser",
    "MinimaxToolParser",
    "KimiK2ToolParser",
    "HunyuanA13BToolParser",
    "Glm4MoeModelToolParser",
    "Qwen3CoderToolParser",
    "Qwen3XMLToolParser",
    "SeedOssToolParser",
    "Step3ToolParser",
    "OpenAIToolParser",
]
```

如果存在，调用工具的模型需要在vllm serve启动的时候添加如下参数：

```
代码块
1   --enable-auto-tool-choice \
2   --tool-call-parser xxx
```

`--tool-call-parser` 参数设置：在当前路径下查询模型对应的py文件，根据**register_module**字段来设置，如：

```python
import ast
import json
import uuid
from collections.abc import Sequence
from typing import Any, Optional, Union

import regex as re

from vllm.entrypoints.openai.protocol import (ChatCompletionRequest,
                                              ChatCompletionToolsParam,
                                              DeltaFunctionCall, DeltaMessage,
                                              DeltaToolCall,
                                              ExtractedToolCallInformation,
                                              FunctionCall, ToolCall)
from vllm.entrypoints.openai.tool_parsers.abstract_tool_parser import (
    ToolParser, ToolParserManager)
from vllm.logger import init_logger
from vllm.transformers_utils.tokenizer import AnyTokenizer

logger = init_logger(__name__)


@ToolParserManager.register_module("qwen3_coder")
class Qwen3CoderToolParser(ToolParser):

    def __init__(self, tokenizer: AnyTokenizer):
        super().__init__(tokenizer)
```

4. 查看reasoning-parser是否支持

Reasoning-parser的支持情况，可在 `vllm/reasoning/__init__.py` 中查询，方法同工具调用，vllm serve启动时需要添加参数如下：

```
代码块
1   --reasoning-parser xxx
```

```
from .abs_reasoning_parsers import ReasoningParser, ReasoningParserManager
from .basic_parsers import BaseThinkingReasoningParser
from .deepseek_r1_reasoning_parser import DeepSeekR1ReasoningParser
from .glm4_moe_reasoning_parser import Glm4MoeModelReasoningParser
from .gptoss_reasoning_parser import GptOssReasoningParser
from .granite_reasoning_parser import GraniteReasoningParser
from .hunyuan_a13b_reasoning_parser import HunyuanA13BReasoningParser
from .mistral_reasoning_parser import MistralReasoningParser
from .qwen3_reasoning_parser import Qwen3ReasoningParser
from .seedoss_reasoning_parser import SeedOSSReasoningParser
from .step3_reasoning_parser import Step3ReasoningParser

__all__ = [
    "ReasoningParser",
    "BaseThinkingReasoningParser",
    "ReasoningParserManager",
    "DeepSeekR1ReasoningParser",
    "GraniteReasoningParser",
    "HunyuanA13BReasoningParser",
    "Qwen3ReasoningParser",
    "Glm4MoeModelReasoningParser",
    "Step3ReasoningParser",
    "MistralReasoningParser",
    "Step3ReasoningParser",
    "GptOssReasoningParser",
    "SeedOSSReasoningParser",
]
```

2.2.3 SGLang推理

SGLang兼容性高，由于目前投入的技术支持资源有限，尚未针对 DCU 进行深度性能调优与加速优化，生产环境与性能敏感场景暂不推荐选用 SGLang。

3 其他

如果依然存在模型无法启动问题，请联系开发人员解决。